Last Hit Under Tower
Fariya Zehrin 24504922

Overview
Each minion is represented by a half-open interval [s_i, f_i). When a minion is committed, the player is occupied throughout that entire time interval, and a collection of minions is feasible if and only if their time intervals are pairwise disjoint (it is allowed for the intervals to touch at their endpoints). The problem then becomes one of finding the largest possible set of pairwise-disjoint intervals. This is known as the Interval Scheduling / Greedy Activity Selection problem, from the CITS3001 greedy-algorithms lectures.

The important point is that the narrative appears to involve an optimisation process over a number of subsets, but a single local rule has been proven to be optimal: always select the compatible window that ends earliest, since ending as early as possible leaves the greatest possible amount of the timeline remaining for future decisions.

Solution
1. Arrange the windows in order according to their finish time f_i.
2. Keep track of last_finish (it starts at -1 since all the s_i are greater than or equal to zero). Go through the windows in order; if s_i is greater than or equal to last_finish, then take the window, increase the answer by one, and set last_finish to f_i.

```
sort windows by f
count = 0; last = -1
for (s, f) in windows:
    if s >= last:
        count += 1; last = f
print(count)
```

Why this is correct (by means of the exchange argument)
Suppose that `g` is the window which has the smallest finish time. Now consider any optimal solution `O` and let `o` denote the window in that solution which finishes earliest. Because `f_g` is less than or equal to `f_o`, we can replace `o` with `g` in `O`, and the resulting solution will still be feasible (since `g` ends no later than `o`, it does not conflict with the other windows) and will have the same size. It follows that at least one optimal solution includes `g`. We can then remove `g` along with all the windows that overlap with it, and apply the argument by induction to the remaining windows.

Data structures
A list of (f, s) pairs, sorted in place according to tuple order, together with two integers.

Why the tempting alternatives fail (these are the intended wrong answers):

- When sorting by the earliest start time (`wa_sort_by_start`), a single extremely long window that begins at 0 blocks everything.
- When selecting on the basis of shortest duration first (`wa_shortest_first`), a short window may span two longer windows that are compatible with each other (for example, the windows `[0,5), [4,6), [5,10)` yield 2, not 1).
- Strict comparison (`wa_strict_touching`): the condition `s > last` rejects touching windows and incorrectly causes chains to be shrunk, for example like `[0,1), [1,2), ...`.
- No sort (`wa_no_sort`): the order in which the input is given is irrelevant.

There are also two slower versions: a dynamic programming solution with time complexity $O(n^2)$ (`tle_quadratic_dp`) and a greedy algorithm based on repeated full rescans (`tle_rescan_greedy`). The dynamic programming approach that is accepted (`dp_bisect`) demonstrates that the same time bound can be achieved using weighted interval scheduling together with binary search.

Complexity

- Time: sorting takes place in O(n log n) time whereas the scan is O(n). When n is 2*10^5, the process runs in less than a second in Python, whereas algorithms with an $O(n^2)$ complexity require on the order of 4*10^10 steps and go beyond the three second limit (this has been confirmed by the tests on the largest inputs).
- Space: O(n) is required for storing the windows (O(1) additional space if the sorting is performed in place).

Test data

The following cases were considered: n equals 1, the extreme values (0 and 10 to the power of 9), identical windows, touching chains, and ties regarding the same start and same finish. With regard to adversarial cases, there is a long window containing a number of short ones (this breaks the sorting by start), repeated "trap triples" (this breaks the shortest-first rule), and an input that is in reverse order (this breaks the no-sort feature). For the randomised cases with n equal to 2 times 10 to the power of 5, both dense short windows and windows of mixed lengths were used. The output was verified by means of a complete brute force approach on 300 randomly chosen small inputs and also by using the dynamic programming solution for all the tests.

AI Use Declaration

AI was used for polishing the idea and not the code.